

Intro to Processor Architecture: Sequential Processor Design

Monishram Selvaraj
2024102076

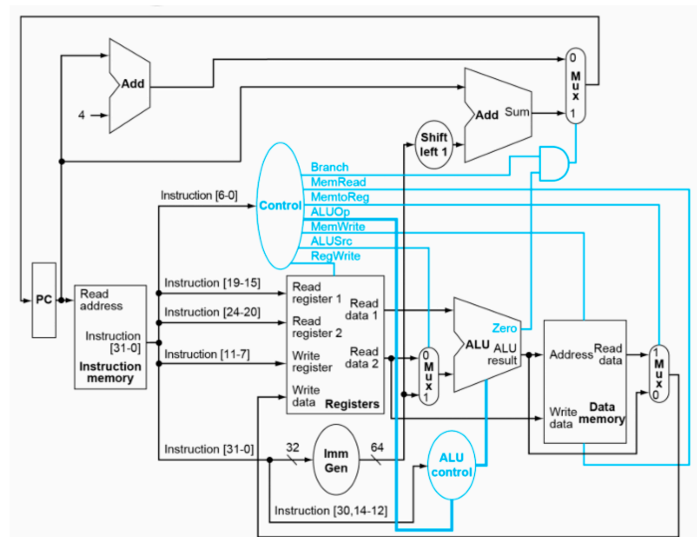
Neel Rajesh
2024102007

Gaurav Reddy
2024112006

Introduction:

The following is the implementation of a sequential 64-bit RISC-V Processor, as a first stage of a larger pipelined processor. The primary goal is to design a processor capable of following RV64I base integer instruction set. The instructions include: add, sub, and, or, beq, ld, sd, addi.

Complete Datapath of the Processor:



Architecture/Components:

- I. **Control Unit:** The control unit takes in the opcode as the input, which are bits [6:0] in the instruction code. The outputs are as follows:
 - A. RegWrite: 1 if the command requires a register write (add, addi, load), 0 for store and branch.
 - B. MemRead: 1 if the command requires loading from memory (load)
 - C. MemWrite: 1 if the command requires writing to memory (store)
 - D. MemtoReg: 1 selects memory output to be written to register, 0 selects ALU result to be written into register.
 - E. ALUSrc: 1 bit signal for 2:1 mux operation:
 1. ALU gets 2 inputs, say A and B, input A is always rs1, and input B is either rs2 or imm value. ALUSrc for choosing between the two.
 - F. Branch
 - G. ALUOp: 2 bit signal
 1. 00 - ADD only (addi, load, store)
 2. 01 - Subtraction for comparison (beq)
 3. 10 - R-type instructions
- II. **ALU Control:** All the operations, add, sub, and, or, share the same opcode (0110011 (R-type)). To solve this problem, the ALU Control gives a signal to the ALU to differentiate between the operations.
 - A. It takes the following as inputs:
 1. ALUOp - from the Control Unit
 2. funct7 bits - [31:25] instruction (7 bits)
 3. funct3 bits - [14:12] instruction (3 bits)
 - B. Gives 4 bits as an output:

1. 0000 - and
2. 0001 - or
3. 0010 - add
4. 0110 - sub

III. ALU (Arithmetic Logic Unit): responsible for performing the actual mathematical and logical operations for the processor. Takes in the 4 bit output from ALU Control (to determine the type of operation), 2 inputs, one from a register, and one from a multiplexer, which decides between another register and immediate value (ALUSrc). The outputs are:

- A. 64 bit calculated output (mathematical operation between two numbers, or a memory address)
- B. Zero flag: 1 bit signal that's high if the output is exactly zero.

IV. PC (Program Counter): 64-bit register that stores the address of the current instruction; increments by 4 to move to the next instruction, or updates to a branch target if there is a jump statement.

V. Register File: Contains 32 64-bit registers; supports two reads or one write at a time. The module prints out the contents itself onto register.txt. The following are the implementation constraints followed:

- A. Big Endianness: The MSB of an address is stored at the smallest memory address, e.g. for a number 0123,
Base+0 -> 01
Base+1 -> 23

VI. Data Memory: performs all the memory read and write operations. Takes memory address as an input,

VII. Instruction Memory: instructions.txt will be present in the same directory of the verilog files, the module will read the file, which contains 1 byte of instruction per line. Since 1 instruction is 32 bits, every 4 lines will constitute one instruction.

- A. We declare a 2D array - reg[7:0]mem[0:IMEMSIZE-1] - where each element is 8 bits long, and the total length of the array is 4096, defined as a macro.
- B. The file is parsed, and each byte is stored as an element in the array.
- C. This module will give the 32 bit (8 byte) instruction as an output, each time the loop is run.

VIII. Immediate Generation: the instruction is fed as an input, and a 64-bit signed extended immediate value is generated accordingly. The 32-bit instruction has an immediate field, which varies according to the type of instruction (I,S, or B).

- A. The module first looks at the opcode of the instruction, which is always located at bits [6:0]. According to the opcode, the instruction type is determined by following the RISC-V ISA set, and the position of the bits are determined.
- B. The immediate value is then signed and extended to 64 bits.

Modular Verification

ALU: Given 4 test cases, which check result and zero flag variable values.

```
VCD info: dumpfile alu_tb.vcd opened for output.
Test 1
A = 000000000000000a
B = 0000000000000005
Result = 000000000000000f
Zero Flag = 0

Status: PASS

Test 2
A = 0000000000000014
B = 0000000000000005
Result = 000000000000000f
Zero Flag = 0

Status: PASS

Test 3
A = 000000000000000a
B = 000000000000000a
Result = 0000000000000000
Zero Flag = 1

Status: PASS

Test 4
A = ff00ff00ff00ff00
B = 0f0f0f0f0f0f0f0f
Result = 0f000f000f000f00
Zero Flag = 0

Status: PASS

Passed 4/4 tests
alu_tb.v:97: $finish called at 50000 (1ps)
```

Immediate Generation: passed +5 and -5 to the registers, checked values after.

```
Test +5  -> imm = 0000000000000005
PASS
Test -5  -> imm = ffffffffbbbbbb
PASS
imm_tb.v:31: $finish called at 20000 (1ps)
```

Register File: Ask to read two registers, verified with actual values

```
Read x2 = 20
PASS: x2 correct
Read x0 = 0
PASS: x0 protected
reg_tb.v:73: $finish called at 42000 (1ps)
```

Control Unit: Checked all values of opcodes, verified what type of instruction it's being determined as, and verified all output values, RegWrite, MemRead, MemWrite, MemtoReg, ALUSrc, Branch, ALUOp.

```
Opcode: 0110011 (0x33) | RegWrite=1 MemRead=0 MemWrite=0 MemtoReg=0 ALUSrc=0 Branch=0 ALUOp=10
PASS: R-type

Opcode: 0010011 (0x13) | RegWrite=1 MemRead=0 MemWrite=0 MemtoReg=0 ALUSrc=1 Branch=0 ALUOp=00
PASS: I-type

Opcode: 0000011 (0x03) | RegWrite=1 MemRead=1 MemWrite=0 MemtoReg=1 ALUSrc=1 Branch=0 ALUOp=00
PASS: Load

Opcode: 0100011 (0x23) | RegWrite=0 MemRead=0 MemWrite=1 MemtoReg=0 ALUSrc=1 Branch=0 ALUOp=00
PASS: Store

Opcode: 1100011 (0x63) | RegWrite=0 MemRead=0 MemWrite=0 MemtoReg=0 ALUSrc=0 Branch=1 ALUOp=01
PASS: Branch

control_tb.v:77: $finish called at 25000 (1ps)
```

ALU Control: Verified all the possible ALUOp values, with the expected output of 4 bit control signal

```
Test:      R-type ADD | ALUOp=10 funct3=000 funct7=0 | Expected=0010 Got=0010
PASS

Test:      R-type SUB | ALUOp=10 funct3=000 funct7=1 | Expected=0110 Got=0110
PASS

Test:      R-type AND | ALUOp=10 funct3=111 funct7=0 | Expected=0000 Got=0000
PASS

Test:      R-type OR  | ALUOp=10 funct3=110 funct7=0 | Expected=0001 Got=0001
PASS

Test:      I-type ADDI | ALUOp=00 funct3=000 funct7=0 | Expected=0010 Got=0010
PASS

Test:      Branch BEQ | ALUOp=01 funct3=000 funct7=0 | Expected=0110 Got=0110
PASS

alu_control_tb.v:57: $finish called at 30000 (1ps)
```

PC and Branch Multiplexers:

```
Test: PC Increment | Expected=4 Got=4  
PASS
```

```
Test: Branch Selection | Expected=100 Got=100  
PASS
```

```
pc_tb.v:61: $finish called at 32000 (1ps)
```

Data Memory: have three test cases, initially writing a value into memory, no errors thrown, reading the same register, verified with output. Third test case involves reading from empty register, shows 0 since all registers are initialised to zero.

```
Test 1: WRITE  
Address=16 Written Data=AAAAAAAAAAAAAAAA  
No read expected in this phase  
PASS if no unintended output change
```

```
Test 2: READ  
Address=16 Expected=AAAAAAAAAAAAAAAA Got=aaaaaaaaaaaaaaaa  
PASS: Read correct
```

```
Test 3: READ from unwritten location  
Expected=0000000000000000 Got=0000000000000000  
PASS: Unwritten memory is zero
```

```
data_tb.v:92: $finish called at 42000 (1ps)
```

Verification of Processor Functioning:

Code 1: Finding a particular number from a given array, exists and sets result to -1 if not found. Contains loop of 7 lines of commands.

```
1  addi x1, x0, 0
2      addi x4, x0, 61      # First element
3      sd x4, 0(x1)
4      addi x4, x0, 60      # Second element
5      sd x4, 8(x1)
6      addi x4, x0, 15      # Third element
7      sd x4, 16(x1)
8      addi x4, x0, 30      # Fourth element
9      sd x4, 24(x1)
10     addi x4, x0, 5        # Fifth element
11     sd x4, 32(x1)
12
13     addi x3, x0, 0        # Initialize index to 0
14     addi x5, x0, 5        # Array length
15     add x7, x1, x0        # Calculate current address
16     addi x2, x0, 69      # Element to be found
17
18     search_loop:
19         beq x3, x5, 28    # If index == length, element not found
20
21         ld x8, 0(x7)      # Load current element
22
23         sub x9, x8, x2     # Compare with search value
24         beq x9, x0, 20    # If equal, element found
25
26         addi x3, x3, 1     # Increment index
27         addi x7, x7, 8     # Increment address
28         beq x0, x0, -24   # Loop back
29
30     not_found:
31         addi x3, x0, -1    # Set result to -1 (not found)
32
33     done:
```

Feb 22 14:39 2026 instructions.txt Page 1

00	13	8e
00	00	63
00	40	00
93	bc	03
03	23	b4
d0	00	03
02	50	40
13	02	24
00	13	04
40	02	b3
b0	40	00
23	b0	04
03	23	8a
c0	00	63
02	00	00
13	01	11
00	93	81
40	00	93
b4	50	00
23	02	83
00	93	83
f0	00	93
02	00	fe
13	83	00
00	b3	04
40	04	e3
b8	50	ff
23	01	f0
01	13	01
e0	00	93
02	51	

Output:

Feb 22 18:43 2026 register file.txt Page 1

0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000045	0000000000000000
ffffffffffffffff	0000000000000000
0000000000000005	0000000000000000
0000000000000005	0000000000000000
0000000000000000	0000000000000000
0000000000000028	0000000000000000
000000000000001e	0000000000000000
ffffffffffffd9	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	53
0000000000000000	

Code 2:

```
1  addi x1, x0, 30      // n = 30
2  addi x2, x0, 0       // sum = 0
3  addi x3, x0, 1       // i = 1 (Loop variable)
4  add  x10, x3, x0      // x10 = 1 (temp)
5
6  Loop:
7  add  x2, x2, x3       // sum += i
8  addi x3, x3, 1        // i++
9  sub  x4, x3, x1       // i - n
10 beq  x4, x10, 8        // if i == n, branch to Exit
11 beq  x0, x0, -16       // branch to Loop (unconditional branch)
12
13 Exit:|
```

Feb 22 18:45 2026 instructions.txt Page 1

01	00	40
e0	01	11
00	85	82
93	33	33
00	00	00
00	31	a2
01	01	04
13	33	63
00	00	fe
10	11	00
01	81	08
93	93	e3

Output:

Feb 22 18:46 2026 register file.txt Page 1

0000000000000000	0000000000000000
000000000000001e	0000000000000000
000000000000001d1	0000000000000000
000000000000001f	0000000000000000
0000000000000001	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000001	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	0000000000000000
0000000000000000	154
0000000000000000	

Challenges Faced:

1. Ensuring correct sign-extension for negative immediate values and branch offsets.
2. Recognising that although addresses are 64-bit, actual memory arrays use limited address bits internally.
3. Observed an offset in total clock cycles because the processor executed one additional fetch cycle after the last valid instruction before the stop condition was triggered, affecting the execution count.